

Arquitectura de Computadores 1

Proyecto 1: Desarrollo de una aplicación para la generación de imágenes y texto

Por

Bagliano Gomez Aldo Alessandro

Profesores: Jeferson González Gómez, Luis Chavarría Zamora

14/04/2025 - 23:59h

Índice

Índice.....	1
Introducción.....	2
Listado de requerimientos del sistema.....	3
Elaboración de opciones de solución al problema.....	4
Comparación de opciones de solución.....	6
Selección de la propuesta final.....	8
Explicación del código de ensamblador.....	9
Conclusiones.....	13

Introducción

En el presente proyecto se desarrolló un sistema que permite aplicar interpolación bilineal sobre imágenes en escala de grises mediante un enfoque híbrido entre alto y bajo nivel. El procesamiento de la imagen se realiza íntegramente en lenguaje ensamblador bajo la arquitectura x86-64, utilizando llamadas al sistema (syscall) para la lectura y escritura de archivos .img binarios. Esta implementación permite escalar un cuadrante de 100×100 píxeles a una imagen interpolada de 400×400 píxeles, utilizando operaciones aritméticas enteras y control manual de memoria, lo cual evidencia el dominio de estructuras de datos en bajo nivel y optimización de rendimiento.

Para la parte de alto nivel se utilizó Python junto a las bibliotecas Tkinter (para la interfaz gráfica), Pillow (para la manipulación de imágenes) y NumPy (para el manejo eficiente de matrices de píxeles). Estas herramientas permitieron crear una interfaz intuitiva en la que el usuario puede cargar una imagen, seleccionar visualmente uno de los 16 cuadrantes disponibles y visualizar el resultado del procesamiento en paralelo. La integración entre Python y el ejecutable en ensamblador se realizó de forma automatizada, permitiendo una comunicación fluida entre ambos niveles. Esta solución combina modularidad, eficiencia, y facilidad de uso, cumpliendo con todos los requerimientos técnicos y funcionales del sistema propuesto.

Listado de requerimientos del sistema

Requerimientos funcionales:

- El sistema debe permitir al usuario cargar una imagen en escala de grises.
- El usuario debe poder seleccionar uno de los 16 cuadrantes (4x4) de la imagen
- El sistema debe generar un archivo .img del cuadrante seleccionado en resolución 100x100 píxeles.
- El código ensamblador debe leer dicho archivo, realizar interpolación bilineal y producir un archivo de salida de 400×400 píxeles (160000 bytes).
- La imagen resultante debe ser visualizada en la interfaz Python junto con la original y el cuadrante seleccionado.
- El flujo completo debe integrarse en un único entorno automatizado.

Requerimientos técnicos:

- Uso de la arquitectura ISA x86 en modo de 64 bits.
- Lenguaje ensamblador para realizar la interpolación bilineal, con llamadas a sistema(syscall).
- El procesamiento de imagen se realiza únicamente en ensamblador, sin intervención de alto nivel.
- Lectura y escritura de archivos binarios planos sin encabezado.
- Compatibilidad con Linux.
- Interfaz de alto nivel desarrollada en Python 3.12 utilizando Tkinter

Estándares , normas y herramientas:

- Uso del formato de archivo .img sin encabezado, interpretado por PIL/Numpy.
- Cumplimiento del estándar POSIX para llamadas a sistema en ensamblador.
- Control de versiones usando Git, siguiendo un flujo de ramas: master, develop y ramas de trabajo.
- Documentación del avance en un repositorio público o privado con acceso para revisión.

Elaboración de opciones de solución al problema

Solución 1:

Descripción:

Python se usa como lenguaje de alto nivel para implementar la interfaz gráfica. Tkinter permite construir interfaces sencillas y multiplataforma. Pillow se encarga de manipular imágenes, y NumPy facilita el trabajo con datos binarios y matrices de píxeles. La integración con el código ensamblador se hace mediante el uso del módulo `os.system()` o `subprocess` para ejecutar el binario.

Ventajas:

- Amplia comunidad y documentación.
- Fácil acceso a los valores de píxeles con NumPy.
- Integración simple con archivos binarios y el sistema de archivos.
- Rápida implementación con bajo esfuerzo de mantenimiento.

Desventajas:

- Poca personalización visual avanzada con Tkinter.
- No ofrece una arquitectura tan robusta para GUI complejas.
- Requiere instalar librerías externas (Pillow, NumPy).

El diagrama para esta solución es el de la **Fig 3** , debido a que es la solución que se escogió

Solución 2:

Descripción:

Utiliza PyQt5 (o su alternativa libre PySide6) para desarrollar una interfaz moderna y visualmente avanzada, con soporte para redimensionamiento, menús y widgets complejos. La biblioteca OpenCV se usa para manejo de imágenes, aunque solo para la visualización y conversión del archivo `.img`. Esta opción permite interfaces más profesionales y con mayor control visual.

Ventajas:

- Interfaces modernas, limpias y profesionales.
- OpenCV tiene herramientas robustas para visualización y conversión de matrices.

- Soporte nativo para eventos y multihilo si se desea mejorar la interacción.

Desventajas:

- Mayor complejidad en la implementación y curva de aprendizaje.
- Necesita más dependencias y configuración.
- Mayor peso en el ejecutable final si se empaqueta.

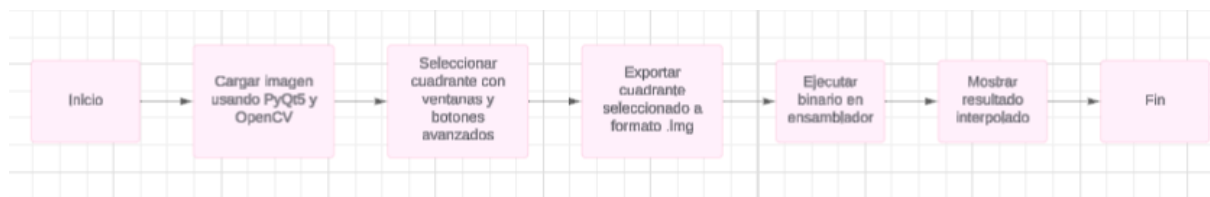


Fig 1:Diagrama de flujo para Solución 2

Solución 3:

Descripción:

En esta alternativa, Java se utiliza para construir la interfaz con Swing o JavaFX. Se implementa un lector binario de archivos .img y se visualiza el contenido como imagen a través de BufferedImage. El binario ensamblador se ejecuta usando ProcessBuilder o Runtime.exec().

Ventajas:

- Interfaz robusta, estructurada, y multiplataforma.
- Buen control del flujo del sistema y manejo de eventos.
- Puede usarse en entornos educativos o empresariales que ya usen Java.

Desventajas:

- No hay librerías nativas para manipular archivos .img, por lo que se debe implementar el manejo de binarios manualmente.
- Menor integración natural con scripts bash o binarios del sistema.
- Java no es ideal para manipulación de imágenes en crudo (requiere más código).



Fig 2:Diagrama de flujo para Solución 3

Comparación de opciones de solución

Tabla 1: Comparación de propuestas de solución.

Criterio	Python (Tkinter + PIL + NumPy)	Python (PyQt5/OpenCV)	Java (Swing/JavaFX)
Complejidad de implementación	Total	Total	Total
Complejidad de implementación	Baja	Media-Alta	Alta
Facilidad de integración con asm	Muy alta (sencilla llamada os.system)	Alta (subprocess o QProcess)	Media (ProcessBuilder, mayor manejo manual)
Manipulación de archivos .img	Muy sencilla con NumPy	Sencilla con OpenCV y numpy	Compleja (requiere lectura binaria personalizada)
Estética / apariencia de GUI	Básica (Tkinter es limitado)	Avanzada (interfaz moderna y fluida)	Media (Swing) a Avanzada (JavaFX)
Portabilidad	Alta (multiplataforma con Python)	Alta (requiere más librerías)	Alta (requiere JRE)
Rendimiento en GUI	Medio (pero suficiente para el proyecto)	Alto (renderizado eficiente con Qt/OpenCV)	Medio (JavaFX tiene buen rendimiento)
Curva de aprendizaje	Baja	Media (estructura más compleja)	Alta (más detallado el manejo binario y gráficos)
Modularidad del código	Alta (código limpio, estructurado)	Alta (MVC posible con PyQt5)	Alta (Java promueve estructuración)
Soporte y comunidad	Muy amplia	Amplia (aunque más técnica)	Amplia, pero menos usada hoy en día

			para GUIs
Estandarización del entorno	Muy buena, estándares abiertos (PIP, PIL)	Muy buena (Qt estándar, OpenCV globalmente usado)	Muy buena (Java estandarizado desde hace décadas)
Compatibilidad con ISA x86	Total (desarrollo en entornos Linux nativos)	Total	Total, pero más fricción con binarios sin encabezado
Aspectos económicos	Muy bajo costo, 100% herramientas libres	Bajo, pero instalación más pesada	Gratuito, pero consumo de recursos mayor
Impacto ambiental	Bajo: ejecuta localmente, sin procesos intensivos	Similar, aunque requiere más dependencias	Mayor uso de memoria/RAM, impacto un poco más alto
Salud y ergonomía	Requiere interacción simple, sin sobrecarga visual	GUI adaptable y agradable, mejor ergonomía	GUI más rígida, potencial sobrecarga visual
Criterios sociales/culturales	Fácil de usar por personas sin experiencia técnica	Interfaz más amigable, accesible	Menor accesibilidad sin conocimientos previos

Selección de la propuesta final

Luego de evaluar las diferentes opciones viables para el desarrollo de la interfaz gráfica del sistema, se seleccionó la propuesta basada en Python utilizando Tkinter, Pillow y NumPy. Esta elección se fundamenta en un análisis técnico integral que considera la facilidad de integración con el código ensamblador, el manejo directo de archivos binarios .img, la portabilidad de la solución, y el tiempo de desarrollo requerido.

La propuesta implementada permitió mantener el enfoque modular y la separación estricta entre el procesamiento (realizado exclusivamente en lenguaje ensamblador x86) y la interfaz de usuario (en Python). Gracias a NumPy, fue posible manipular los datos en bruto de las imágenes con precisión y eficiencia. Pillow ofrece soporte nativo para trabajar en escala de grises y recortar imágenes, mientras que Tkinter permitió implementar una GUI funcional sin necesidad de bibliotecas externas complejas.

Si bien otras propuestas como PyQt5/OpenCV o Java Swing ofrecían beneficios adicionales en cuanto a estética o estructura, introducían una mayor complejidad de desarrollo y no aportan ventajas determinantes dentro del contexto del proyecto, donde la prioridad era facilitar la conexión directa entre la interfaz y el procesamiento ensamblador.

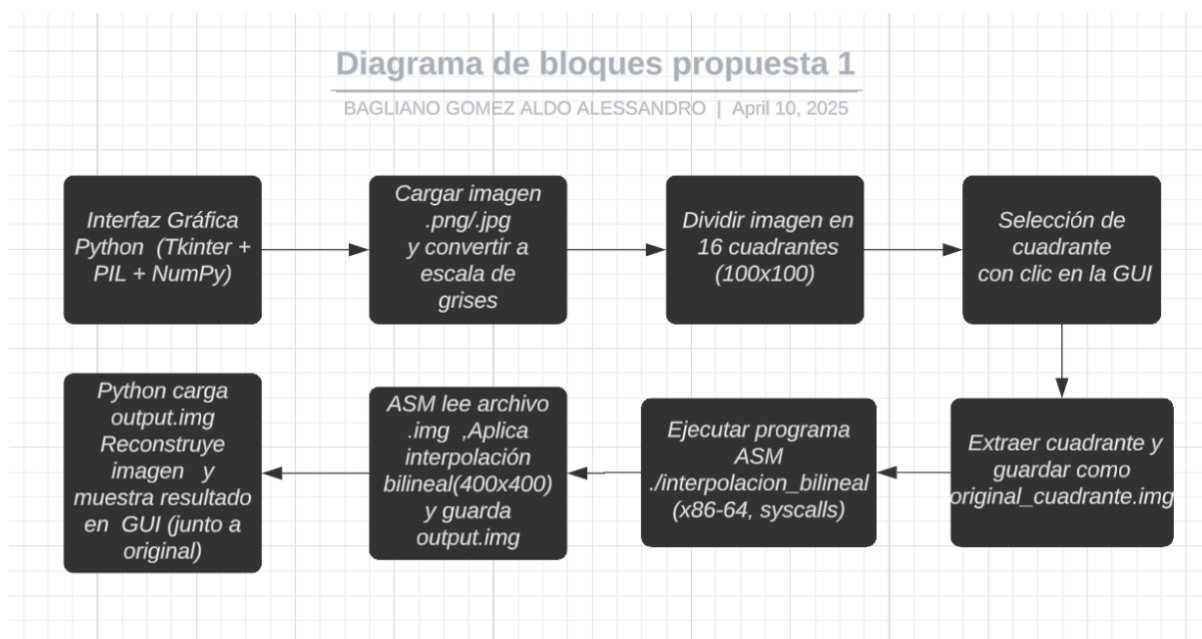


Fig 3:Diagrama de bloques del sistema.

A continuación se presenta en la **Fig 4** la interfaz gráfica desarrollada para la propuesta 1 , en la cual se puede apreciar la imagen cuadriculada al lado izquierdo junto con la imagen interpolada del

cuadrante elegido al lado derecho.

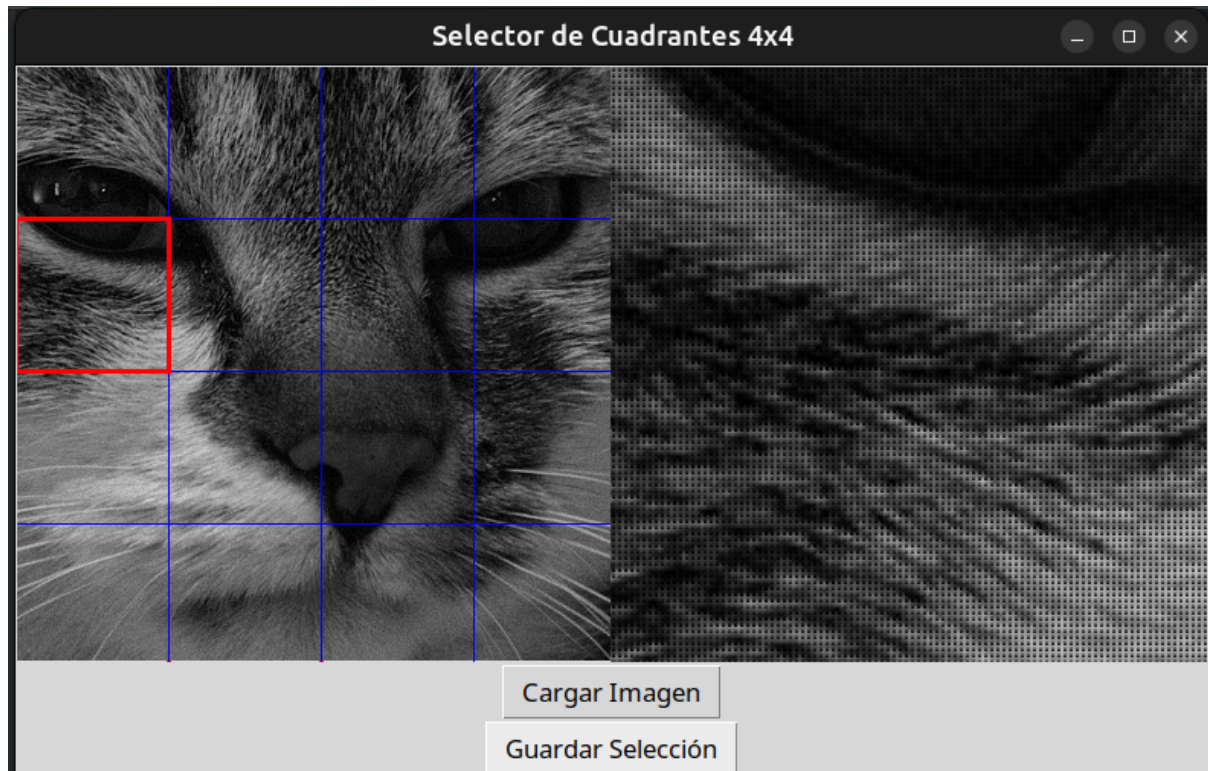


Fig 4: Interfaz en python del sistema.

Explicación del código de ensamblador

Para hacer la explicación del código se va a hacer mediante pseudocódigo

1. Estructuras de Datos Principales

CONSTANTES:

```
IN_SIZE = 10000    // 100x100 pixels
OUT_SIZE = 160000  // 400x400 pixels
WIDTH = 100        // Ancho entrada
OUT_WIDTH = 400     // Ancho salida
SCALE_FACTOR = 4    // Factor de escala
```

VARIABLES:

```
input_file = "original_cuadrante.img"
output_file = "output.img"
input_buf[IN_SIZE]    // Buffer entrada
output_buf[OUT_SIZE]  // Buffer salida
fd_in, fd_out         // Descriptores de archivo
```

2. Flujo Principal

PROGRAMA PRINCIPAL:

```
LEER_ARCHIVO_ENTRADA()
```

```
INTERPOLAR_IMAGEN()
ESCRIBIR_ARCHIVO_SALIDA()
SALIR_CON_EXITO()
```

3. Lectura del Archivo de Entrada

FUNCIÓN LEER_ARCHIVO_ENTRADA():

```
ABRIR archivo input_file en modo lectura → fd_in
SI falla ABRIR → MOSTRAR_ERROR()
```

```
LEER fd_in en input_buf, tamaño IN_SIZE
SI bytes leídos ≠ IN_SIZE → MOSTRAR_ERROR_TAMAÑO()
```

```
CERRAR fd_in
```

4. Algoritmo de Interpolación

4.1 Fase 1: Interpolación Horizontal/Vertical

FUNCIÓN INTERPOLAR_IMAGEN():

```
PARA cada fila y DESDE 0 HASTA 99:
PARA cada columna x DESDE 0 HASTA 99:
```

```
// Calcular posición en salida (escala 4x)
```

```
pos_salida = (y*4)*400 + (x*4)
```

```
// Obtener píxeles 2x2 originales
```

```
P00 = input_buf[y*100 + x]
```

```
P10 = (x == 99) ? P00 : input_buf[y*100 + x+1]
```

```
P01 = (y == 99) ? P00 : input_buf[(y+1)*100 + x]
```

```
P11 = (x == 99 o y == 99) ? P01 : input_buf[(y+1)*100 + x+1]
```

```
// Interpolación horizontales (primera fila)
```

```
a1 = (3*P00 + 1*P10) / 4
```

```
a2 = (2*P00 + 2*P10) / 4
```

```
a3 = (1*P00 + 3*P10) / 4
```

```
// Interpolación verticales (primera columna)
```

```
c1 = (3*P00 + 1*P01) / 4
```

```
c2 = (2*P00 + 2*P01) / 4
```

```
c3 = (1*P00 + 3*P01) / 4
```

```
// Guardar píxeles calculados
```

```
output_buf[pos_salida] = P00
```

```
output_buf[pos_salida+1] = a1
```

```
output_buf[pos_salida+2] = a2
```

```
output_buf[pos_salida+3] = P10
```

```
output_buf[pos_salida+400] = c1
```

```
output_buf[pos_salida+800] = c2
output_buf[pos_salida+1200] = P01
output_buf[pos_salida+1203] = P11
```

4.2 Fase 2: Interpolación de Píxeles Centrales

```
// Procesar bloques 4x4 para calcular píxeles centrales
PARA cada fila y DESDE 0 HASTA 396, PASO 4:
PARA cada columna x DESDE 0 HASTA 396, PASO 4:
```

```
pos = y*400 + x
```

```
// Obtener esquinas del bloque 4x4
P00 = output_buf[pos]
P10 = output_buf[pos+3]
P01 = output_buf[pos+1200]
P11 = output_buf[pos+1203]
```

```
// Interpolar píxeles intermedios
PARA cada fila_intermedia DESDE 1 HASTA 3:
PARA cada col_intermedia DESDE 1 HASTA 3:
```

```
    // Pesos basados en posición relativa
    peso_arriba = 4 - fila_intermedia
    peso_abajo = fila_intermedia
    peso_izq = 4 - col_intermedia
    peso_der = col_intermedia
```

```
    // Obtener píxeles superiores/inferiores
    superior = output_buf[pos + col_intermedia]
    inferior = output_buf[pos + 1200 + col_intermedia]
```

```
    // Calcular nuevo píxel
    pixel = (peso_arriba*superior + peso_abajo*inferior) / 4
```

```
    // Guardar resultado
    pos_final = pos + (fila_intermedia*400) + col_intermedia
    output_buf[pos_final] = pixel
```

5. Escritura del Archivo de Salida

FUNCIÓN ESCRIBIR_ARCHIVO_SALIDA():

```
    CREAR/ABRIR archivo output_file en modo escritura → fd_out
    SI falla → MOSTRAR_ERROR()
```

```
    ESCRIBIR output_buf en fd_out, tamaño OUT_SIZE
    SI bytes escritos ≠ OUT_SIZE → MOSTRAR_ERROR()
```

```
    CERRAR fd_out
```

6. Manejo de Errores

```
FUNCIÓN MOSTRAR_ERROR():  
    IMPRIMIR "File operation failed"  
    SALIR_CON_FALLA()
```

```
FUNCIÓN MOSTRAR_ERROR_TAMAÑO():  
    IMPRIMIR "Input file size incorrect"  
    SALIR_CON_FALLA()
```

```
FUNCIÓN SALIR_CON_EXITO():  
    TERMINAR_PROGRAMA(0)
```

```
FUNCIÓN SALIR_CON_FALLA():  
    TERMINAR_PROGRAMA(1)
```

Conclusiones

Este proyecto permitió consolidar conocimientos sobre la interacción entre software de alto y bajo nivel. Se logró una integración funcional entre una interfaz desarrollada en Python y un módulo de procesamiento implementado en lenguaje ensamblador x86, demostrando la viabilidad de este tipo de arquitecturas mixtas en aplicaciones reales.

La implementación de la interpolación bilineal en lenguaje ensamblador brindó una oportunidad valiosa para aplicar conceptos de direccionamiento, manipulación de registros, operaciones con memoria y llamadas al sistema en un entorno práctico.

La elección del ISA de x86 fue acertada debido a que permite hacer una depuración nativa desde linux junto con herramientas como lo es gbd , lo que acelera el ciclo de desarrollo del proyecto además de que es un poco más sencillo de usar que con los ISA de ARM o Risc V en especial.

El uso de ensamblador para desarrollar el algoritmo de interpolación bilineal otorgó un control total sobre la manipulación de los datos de imagen, permitiendo una comprensión más profunda del tratamiento de píxeles a nivel de hardware. Esta solución también garantiza eficiencia y precisión en el procesamiento.